Case 2: Real-Time Medical Evaluation Platform

Monitoring and Performance Engineering

Your Name
Your Degree, Your University

November 3, 2026

Abstract

A clinic monitors the vital signs of elderly patients in real time. When a device failed, the platform did not issue an alert in time because it recalculated every value from raw data and fell behind under load. This report analyses the incident with Bentley's rules for performance, answers the three questions of the case, and backs the argument with a small Java implementation: a benchmark of late versus early binding and a replay of the incident with a watchdog for silent devices. Recomputing a 5-minute average from stored history cost 4.5 µs with 10 000 stored readings and 5.42 ms with 5 000 000, while the incremental (early-binding) update stayed between 11.7 and 20.8 ns regardless of history size. All code is available in the accompanying repository.

1 Scenario and diagnosis

A clinic uses a platform that continuously receives data from wearable sensors, stores it in a real-time database (a streaming data system) and alerts relatives or medical staff when values fall outside normal parameters. Recently a device failed and the system did not alert in time: it recalculated every value constantly, and the large data volume caused delays.

Using the vocabulary of the course [4, 3], the incident has three causes:

- **"Do it, but don't do it again".** Values were recomputed from raw data each time. When an operation must be done, its result should be stored (memoization or caching).
- **Early binding was missing.** The work for critical indicators could have been done earlier, at ingestion, instead of during the emergency.
- **A monitoring gap.** A device that fails silently produces no data, so no threshold rule fires. "No data" was treated as "no problem".

2 How can the data flow be optimized for emergencies?

Bentley's rules [1, 2] apply directly to the pipeline (Table 1).

Table 1: Bentley's rules applied to the monitoring pipeline.

Rule	Application to the platform
Don't do it	Do not recompute from raw history. Discard invalid or noisy readings at the sensor or gateway.

Rule	Application to the platform
Problem simplification	The alert only needs to know whether a value is out of range, not a full analysis. Keep a lightweight alert path (threshold checks, short moving windows) separate from heavy analytics and reports.
Relentless suspicion	Question every operation on the critical path. Anything not needed to decide "alert or not" moves off it.
Do it less	Use incremental updates (update the running average with each new reading) instead of rescanning data. Partition streams by patient so one patient's load does not affect another's.
Do it better	Use parallelism (one processing unit per patient partition) and data structures suited to the access pattern (fixed-size sliding windows over primitive arrays).
Ordering tests	Evaluate the cheapest and most critical checks first (e.g. heart rate or SpO_2 beyond hard limits) and stop as soon as an alert triggers.
Hoisting	Compute per-patient thresholds once, outside the per-reading loop.

Fast path and slow path. Figure 1 separates the real-time path (sensor, gateway, broker, stream processor, alert engine, notification), which does only light work, from the analytical path (time-series database and cold storage), which never blocks alerts. The in-memory store keeps the latest state and an alert-status flag per patient.

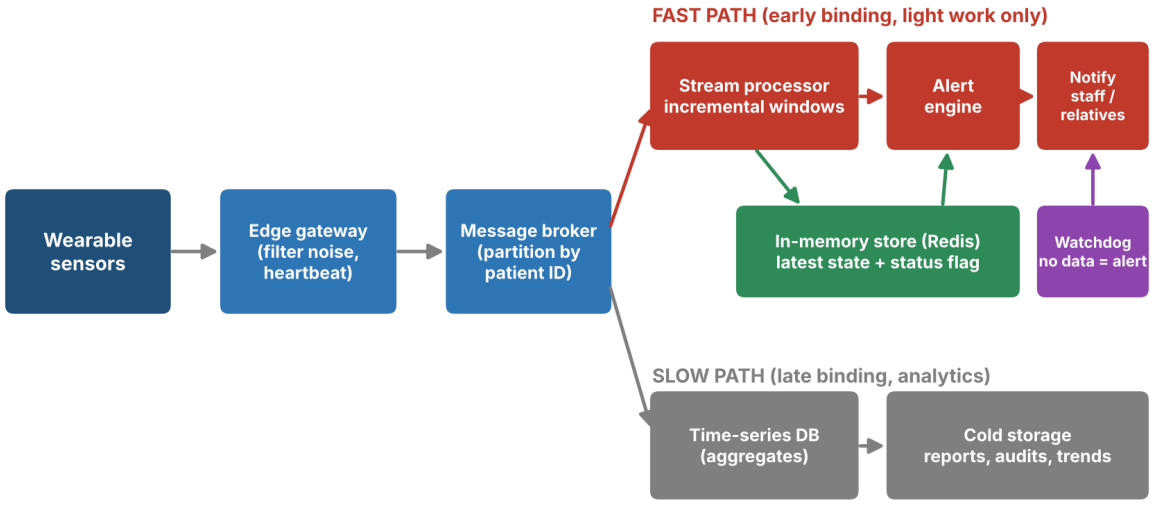


Figure 1: Proposed data flow. The fast path (red) uses early binding; the slow path (grey) keeps late binding for analytics.

Monitoring the monitor. A *watchdog* receives a heartbeat with every reading and raises a `DEVICE_OFFLINE` alert when none arrives within a timeout. Pipeline lag should also be tracked, so that a delay is itself an alert. Redundant brokers, several alert channels (SMS, call, app) and an escalation chain when nobody acknowledges complete the design.

Replay of the incident. The Java program `FailureSimulation` generates one reading per second for three patients over 600 s (Figure 2). Patient 2 has a tachycardia episode, and the 5-second average (early binding) crosses 120 bpm, raising a threshold alert at $t = 204$ s. The

device of patient 3 stops at $t = 300\,\text{s}$. No threshold rule can fire without data, but the watchdog (timeout 10 s) raises `DEVICE_OFFLINE` at $t = 310\,\text{s}$, 10 seconds after the last reading.



Figure 2: Simulated replay. Patient 2 triggers a threshold alert; the silent failure of patient 3 is caught only because of the watchdog.

3 Advantages and disadvantages of early binding

Early binding means performing computations earlier to avoid redundant work later [4]. In this context, indicators, aggregates and alert evaluations are computed and fixed at ingestion time, before anyone queries them. The trade-off is memory for time. The opposite idea, *lazy evaluation*, delays computation until the value is required; it suits rarely used reports but not critical alerts, where waiting is dangerous.

Table 2: Early binding in a real-time monitoring context.

Advantages	Disadvantages
Alerts are fast and predictable: the indicator is already computed when needed.	Uses more memory and storage for derived values.
Eliminates repeated work, so it scales as data volume grows.	Less flexible: a new indicator or changed formula means recomputing, and past data may need reprocessing.
Many consumers (alerts, dashboard, reports) reuse the same precomputed value.	Costs work on every reading, even if nobody queries the result.
The system stays responsive when it matters most: emergencies.	Stale values are a risk: a cached value that is not updated correctly could hide a real emergency.
	More complex design (rule versioning, late and out-of-order data).

Experiment. To quantify the benefit, the Java program compares two implementations that return the same 5-minute average heart rate (a sanity check verifies they agree before timing). `LateBindingAverage` stores all readings and rescans the history on each request. `EarlyBindingAverage` keeps a running sum in a circular buffer of primitive arrays and updates it as each reading arrives. Each value is the median of nine timed trials after a discarded warm-up (Java 21.0.12.1, 2 CPU cores; absolute values depend on the machine).

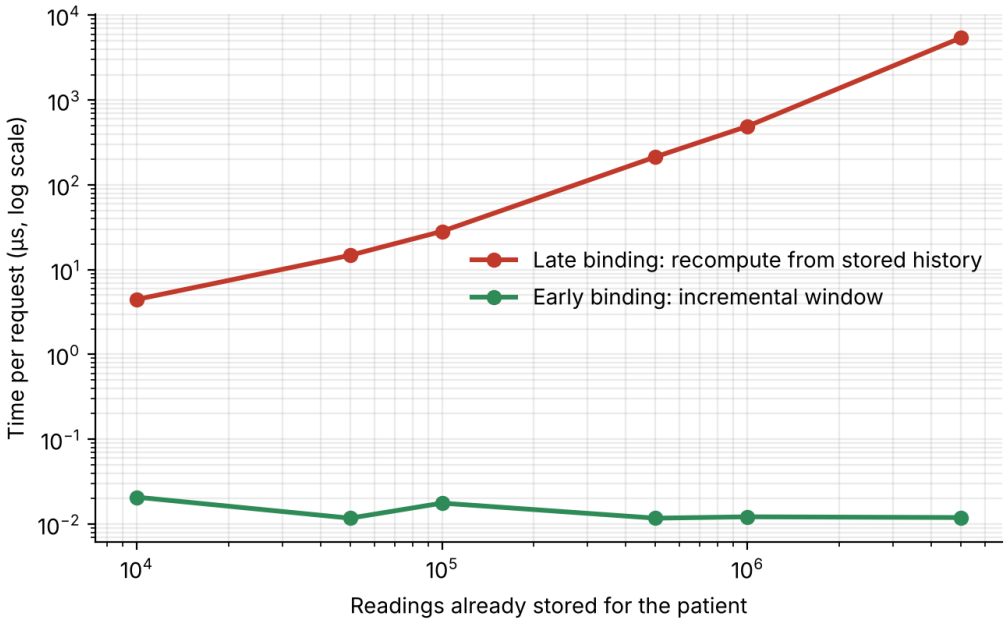


Figure 3: Measured time per request (log–log). The late-binding cost grows linearly with the stored history; the early-binding cost is flat.

Table 3: Measured results.

Stored readings	Late binding (µs)	Early binding (ns)	Speed-up (×)	Cores needed (late)*
10 000	4.5	20.8	210	0.004
50 000	14.8	11.7	1 300	0.015
100 000	28.3	17.7	1 600	0.028
500 000	214.0	11.7	18 000	0.214
1 000 000	491.2	12.2	40 000	0.491
5 000 000	5 419.8	11.9	450 000	5.420

* Derived, not measured: assumes 1 000 patients sending one reading per second, one recomputation per reading, on one thread.

Reading the results. For a single patient the absolute cost of late binding is small at moderate history sizes: Java with primitive arrays scans millions of values in milliseconds. The problem is how the cost scales. It grows linearly with the history and is paid on every request, for every patient and every metric. With the assumptions of Table 3, recomputation needs more than one full CPU core once the history reaches a few million readings (about 58 days of 1 Hz data), whereas the incremental update stays negligible. This is the mechanism behind the delays reported in the case. The experiment is deliberately simple (one metric, synthetic data, a single thread) and shows the trend, not the performance of a production system.

Conclusion. Use early binding for a small set of critical, well-defined indicators (heart rate, SpO_2, blood pressure, trends). Keep late binding or lazy evaluation for exploratory and historical analysis. A hybrid approach is best.

Optimization ethics. Knuth warns that premature optimization is the root of all evil [6], and Jackson's first rule of optimization is "Don't do it" [5]. Here the optimization is justified because the failure revealed a real problem: delays under load. Even so, the best practices of the course still apply: profile first to confirm that recomputation is the bottleneck, keep the alert logic correct and clear before making it fast, and measure the improvement afterwards (which is what the benchmark does).

4 How can pre-computed values be stored for fast retrieval?

Storing precomputed values is memoization and caching applied to the platform. Different layers serve different purposes (Table 4).

Table 4: Storage layers for precomputed and raw data.

Layer	**Purpose**
In-memory key-value store (e.g. Redis)	Latest state per patient with instant lookup.
Stream-processor state	Sliding windows and running aggregates kept next to the computation.
Time-series database with continuous aggregates	Per-minute or hourly summaries for dashboards and trends.
Cold storage (data lake or warehouse)	Raw data kept only for audits and analysis.

Example record.

```
key: patient:1042:vitals:latest
value: { heart_rate: 118, spo2: 91,
        hr_avg_5min: 104, trend: "rising",
        status: "WARNING", rule_version: 3,
        last_update: "2026-11-02T19:20:05Z" }
```

Design points.

- **Key by patient, metric and window**, so retrieval is a direct lookup and not a scan (data-structure optimization, memory layout and locality).
- **Store the alert status as a flag**, so the notifier reads a single value.
- **Keep a timestamp on each value.** If it is too old, treat it as a possible device failure; this also removes the stale-cache risk.
- **Version the rules.** Thresholds differ per patient (e.g. chronic conditions), so store which rule version produced each value.
- **Plan for failure:** use replication and snapshots, and be able to rebuild derived values from the raw stream.

5 Implementation

The repository contains the Java program (package `monitoring`) and the Python script that produces all figures and tables from its output. The early-binding structure and the watchdog are shown in Listings 1 and 2; the remaining classes (`LateBindingAverage`, `Benchmark`, `FailureSimulation`, `Main`) are in the repository. The early-binding class applies three rules of the course: it stores the running sum (*don't do it again*), touches each reading once on entry and once on exit (*do it less*), and uses primitive arrays in a circular buffer (*primitives versus objects*, memory locality).

Listing 1: Incremental sliding-window average (early binding).

```
1  package monitoring;
2
3  /**
4   * Early binding: the moving average is updated incrementally AT INGESTION,
5   * so reading it later costs O(1) no matter how much history exists.
6   *
7   * Bentley rules applied:
8   * - "Do it, but don't do it again": the running sum is stored and reused.
9   * - "Do it less": each reading is added once and removed once.
10  * - Primitive arrays (not objects) and a circular buffer for memory locality.
11  *
12  * Trade-off (memory for time): only the readings inside the window are kept.
13  */
14 public final class EarlyBindingAverage {
15     private final long windowSec;
16     private final long[] ts;
17     private final double[] vals;
18     private int head = 0;
19     private int size = 0;
20     private double sum = 0.0;
21
22     /**
23      * @param windowSec window length in seconds (e.g. 300 for a 5-minute window)
24      * @param capacity maximum readings held; must exceed windowSec * sampling rate
25      */
```

```
    public EarlyBindingAverage(long windowSec, int capacity) {
        this.windowSec = windowSec;
        this.ts = new long[capacity];
        this.vals = new double[capacity];
    }

    /** Ingest one reading and return the up-to-date window average. */
    public double add(long t, double v) {
        while (size > 0 && ts[head] < t - windowSec) {
            evictOldest();
        }
        if (size == ts.length) { // buffer full: drop the oldest reading
            evictOldest();
        }
        int tail = (head + size) % ts.length;
        ts[tail] = t;
        vals[tail] = v;
        size++;
        sum += v;
        return sum / size;
    }

    /** O(1) read of the precomputed value. */
    public double average() {
        return size == 0 ? Double.NaN : sum / size;
    }

    private void evictOldest() {
        sum -= vals[head];
        head = (head + 1) % ts.length;
        size--;
    }
}
```

Listing 2: Watchdog that detects silent devices.

```
package monitoring;

/**
 * "Monitoring the monitor": raises a DEVICE_OFFLINE alert when a patient's
 * device has been silent for longer than the timeout. Without this, a failed
 * sensor produces no data, so no threshold rule ever fires ("no data" would be
 * mistaken for "no problem").
 */
public final class Watchdog {
    private final long timeoutSec;
    private final long[] lastSeen;
    private final boolean[] alerted;

    public Watchdog(int patients, long timeoutSec) {
        this.timeoutSec = timeoutSec;
        this.lastSeen = new long[patients];
        this.alerted = new boolean[patients];
    }

    /** Call whenever a reading arrives (the heartbeat). */
    public void heartbeat(int patient, long t) {
        lastSeen[patient] = t;
        alerted[patient] = false;
    }

    /** @return true exactly once per silence episode, when the timeout is exceeded. */
    public boolean check(int patient, long now) {
        if (!alerted[patient] && now - lastSeen[patient] > timeoutSec) {
```

```
            alerted[patient] = true;
            return true;
        }
        return false;
    }

    public long lastSeen(int patient) {
        return lastSeen[patient];
    }
}
```

Repository: https://github.com/YOUR-USER/YOUR-REPO

6 Conclusion

The incident shows that recalculating every value on demand does not scale in a real-time medical system. Applying Bentley's rules, specifically *Do it, but don't do it again* (caching) and *Early binding*, critical indicators should be computed incrementally at ingestion and stored in a fast in-memory or time-series layer, separate from the analytical path. The experiment shows the cost of recomputation growing with history while the incremental update stays constant, and the replay shows that a watchdog closes the monitoring gap for silent devices. The costs are extra memory, less flexibility and the risk of stale data, which timestamps and rule versioning mitigate. Consistent with the optimization ethics of Knuth and Jackson, the changes should be justified by measurement and must keep the alert logic correct.

References

[1] Jon Louis Bentley. *Writing Efficient Programs*. Prentice-Hall, 1982.

[2] Jon Louis Bentley. *Programming Pearls*. Addison-Wesley, 1986.

[3] Course material. Applications – monitoring and performance engineering, 2026. Application notes provided by the instructor.

[4] Course material. Lectures – monitoring and performance engineering, 2026. Lecture notes provided by the instructor.

[5] Michael A. Jackson. *Principles of Program Design*. Academic Press, 1975.

[6] Donald E. Knuth. Structured programming with `go to` statements. *ACM Computing Surveys*, 6(4):261–301, 1974.